

Module/môn: Cấu trúc dữ liệu và giải thuật

Bài assignment này đòi hỏi sinh viên dành khoảng 24h làm để hoàn thành.

Tương ứng với mục tiêu môn học: A, B.

Gian lận

Gian lận là hình thức lấy bài làm của người khác và sử dụng như là mình làm ra. Bao gồm: copy từ web/sách mà không ghi nguồn; gửi bài làm chung nhưng nộp như sản phẩm cá nhân; copy bài của sinh viên khác; mua/ăn cắp bài của người khác.

Mọi tài nguyên copy hay điều chế từ nguồn nào (Internet, sách, slide...) phải đặt trong cặp dấu nháy kép và in nghiêng, kèm thông tin tham khảo đầy đủ.

Bài làm sẽ được đưa vào phần mềm kiểm tra gian lận. Mọi hành vi cố tình lách hệ thống phát hiện gian lận sẽ bị xử lý theo quy định Nhà trường.

Quy định nộp bài

- Nộp bản mềm trước 23h59 (giờ địa phương) ngày hết hạn trên LMS.
- Được upload đề nhiều lần, bài chấm là bản cuối cùng.
- File văn bản ở dạng gốc (docx, py, ipynb...), không nộp dạng ảnh chụp.
- Nén toàn bộ sản phẩm vào 1 file .zip, dung lượng tuân thủ giới hạn hệ thống.
- Không nộp file nhiễm virus, không đặt mật khẩu.
- Không cần gửi lại file đề bài assignment này.

Quy định đánh giá

- Không có bài trên hệ thống → 0 điểm assignment.
- Nộp muộn: trong 1 tuần có thể xin gia hạn; nếu không được duyệt → giữ 0 điểm.
- Nếu phát hiện gian lận trước/sau khi chấm điểm → bài bị hủy, xử lý kỷ luật.

Assignment

Mục tiêu cụ thể

- Vận dụng **Dijkstra** và **Kruskal + Union-Find** để giải quyết các bài toán mô hình hóa mạng giao hàng: tìm tuyến vận chuyển rẻ nhất, xây dựng mạng kết nối tối thiểu.
- Áp dụng **hashing** (dict, set, rolling hash) để tra cứu nhanh thông tin đơn hàng, nhóm đơn hàng tương tự, kiểm tra chuỗi ký tự (mã giảm giá, pattern).
- Áp dụng **dynamic programming** (memoization, bottom-up, 2D/1D) để tối ưu lựa chọn gói khuyến mãi sản phẩm (knapsack 0/1) và các bài toán kiểu stairs/chuỗi quyết định.
- Thiết kế code thành các **module giải thuật** độc lập, dễ tái sử dụng: routing, hashing_tools, promo_optimizer, kèm test nhỏ, giống cách các khóa DSA thiết kế “algo library” cho project cuối kỳ.

Công cụ cần có

- Ngôn ngữ: Python 3.x.
- IDE: VS Code / PyCharm / Jupyter / Colab.
- Thư viện chuẩn: heapq, collections, math, random, itertools.
- Không dùng thư viện giải thuật cao cấp (networkx, numpy) để làm thay thuật toán cốt lõi.

Tham khảo

- Slide bài 1–8 của môn.
- Các nguồn DSA phổ biến: GeeksforGeeks, cp-algorithms, LeetCode Patterns (hashing, graph, DP).

ĐẶT VẤN ĐỀ

Bạn đang tham gia vào đội phát triển hệ thống hậu cần cho sàn thương mại điện tử **POLY-SHIP**. Hệ thống cần:

- Tính toán **tuyến giao hàng ngắn nhất** giữa các kho và điểm giao (graph + Dijkstra).
- Thiết kế **mạng liên kết các kho** với chi phí lắp đặt cổng trung chuyển thấp (MST + Kruskal + DSU).
- Quản lý **hàng trăm nghìn đơn hàng**: tra cứu, nhóm đơn hàng tương tự, tìm pattern trong chuỗi (hashing).
- Đề xuất **combo khuyến mãi** cho khách (gói sản phẩm trong giới hạn ngân sách nhưng giá trị cao nhất: knapsack 0/1).

Bạn cần xây dựng một project Python dạng “**thư viện giải thuật + demo**”, không phải UI đẹp, mà tập trung vào **đúng giải thuật và cấu trúc dữ liệu**.

YÊU CẦU

Gợi ý cấu trúc dự án:

- `routing.py`: thuật toán Dijkstra, Kruskal + DSU.
- `hashing_tools.py`: hash table demo, các bài hashing tổng hợp (group anagrams, longest consecutive, subarray sum = k, rolling hash).
- `promo_optimizer.py`: dynamic programming – fibonacci, climbing stairs, knapsack 0/1 (2D/1D).
- `main_assignment.py`: menu chạy thử các case thực tế của POLY-SHIP.

Sinh viên có thể tự đặt tên khác, nhưng nội dung phải tương đương.

Phần 1 – Routing giữa các kho

1.1. Mạng kho và điểm giao (Shortest Path – Dijkstra)

Bối cảnh:

- POLY-SHIP có hệ thống kho và điểm giao, mỗi cung đường có chi phí (thời gian/tiền).
- Bạn được cung cấp 1 file dữ liệu đơn giản (có thể là CSV hoặc hard-code trong file code) mô tả:

u, v, cost (vô hướng).

Yêu cầu 1.1:

Trong `routing.py`, cài đặt:

1. `build_graph(edges)`
 - Input: list (u, v, cost).
 - Output: adjacency list `graph = {u: [(v, cost), ...], ...}`.
2. `dijkstra(graph, source)`
 - Giống buổi 6: dùng `heapq`, trả về `dist`, `parent`.
3. `shortest_route(graph, source, target)`
 - Gọi Dijkstra và trả về:
 - Chi phí đường đi ngắn nhất.
 - Danh sách các điểm đi qua (route).
4. `demo_routing_shortest_path()`
 - Tạo 1 đồ thị “mạng kho” (5–8 đỉnh: WH1, WH2, HCM, HN, DN...).
 - Cho user nhập `source`, `target` (hoặc dùng test cứng).
 - In: route + tổng chi phí.

1.2. Thiết kế mạng kho tối thiểu (MST – Kruskal + DSU)

POLY-SHIP muốn lắp đặt **đường truyền nội bộ riêng** giữa các kho chính, sao cho:

- Tất cả kho kết nối (liên thông).

- Tổng chi phí cáp/đường truyền là thấp nhất $\rightarrow$ MST.

Yêu cầu 1.2:

Trong `routing.py`, cài đặt:

1. DSU (Union-Find): `make_set`, `find`, `union` với path compression + union by size.
2. `kruskal_mst(vertices, edges)`
 - Input: danh sách kho, danh sách cạnh (chi phí lắp đặt).
 - Output: list cạnh trong MST, tổng chi phí.
3. `demo_mst_network()`
 - Dùng cùng tập kho như trên.
 - In:
 - Danh sách đường truyền được chọn.
 - Tổng chi phí lắp đặt.
 - Nhận xét: “Đây là bộ khung tối thiểu, các tuyến giao hàng chi tiết dùng Dijkstra trên mạng này.”.

Phần 2 – Hashing cho đơn hàng và dữ liệu

Trong `hashing_tools.py`.

2.1. Hash table dùng cho tra cứu đơn hàng

Bài toán:

- Mỗi đơn hàng có `order_id` (string).
- Cần tra nhanh “đơn hàng X có tồn tại không?”, “thông tin chi tiết”.

Yêu cầu 2.1:

1. Implement `OrderHashTable` (separate chaining):

- API:
 - `insert(order_id, order_data)`
 - `get(order_id)`
 - `remove(order_id)`

2. `demo_order_hash_table()`

- Tạo vài đơn hàng mẫu (id + info).
- Cho test insert, get, remove, in ra màn hình.

2.2. Group anagrams – nhóm mã sản phẩm “na ná nhau”

Giả sử hệ thống có list mã coupon: "SAVE10", "AVES10", ...

Yêu cầu:

- Viết hàm `group_coupon_anagrams(codes)` dùng hash map để nhóm các mã là anagram của nhau.

Demo:

- In từng nhóm mã, giải thích ứng dụng (phát hiện mã “khác tên nhưng giống cấu trúc”).

2.3. Longest consecutive – ngày giao liên tiếp

Giả sử bạn có list ngày (dưới dạng integer day-index) mà hệ thống ghi nhận đơn hàng.

Yêu cầu:

- Viết `longest_consecutive_days(days)` tìm chuỗi ngày liên tiếp dài nhất (các event xảy ra liên tục).

Ứng dụng gợi ý:

- Dùng để phát hiện “chuỗi ngày cao điểm” liên tiếp.

2.4. Subarray sum = k – kiểm tra doanh thu theo chuỗi ngày

Bạn có list doanh thu từng ngày, muốn đếm số khoảng ngày liên tiếp có tổng doanh thu đúng $= k$.

Yêu cầu:

- Viết `count_revenue_windows(revenues, k)` dùng prefix sum + hash map.

Demo:

- In số đoạn con có tổng $= k$ với vài bộ test.

2.5. Rolling hash – tìm mã giảm giá trong log

Giả sử bạn có chuỗi log dài, và muốn tìm pattern (một mã coupon hoặc chuỗi ký tự đặc biệt).

Yêu cầu:

- Cài `rolling_hash_search(text, pattern)` kiểu Rabin–Karp đơn giản (như buổi 7.2).
- Viết `demo_rolling_coupon_search()` với một text và vài pattern.

Phần 3 – Dynamic Programming cho gợi ý khuyến mãi

Trong `promo_optimizer.py`.

3.1. Ôn lại DP nhẹ: stairs & fibonacci

1. `fib_tab(n)` hoặc `fib_memo(n)`.
2. `climb_stairs(n)` – số cách lên n bậc (1 hoặc 2 bước).

`demo_dp_basics()`:

- In giá trị 2 hàm với n nhỏ, nhắc lại tư duy state/transition.

3.2. Khuyến mãi combo (knapsack 0/1)

Bối cảnh:

- Có m sản phẩm muốn đưa vào chương trình combo.

- Mỗi sản phẩm i có:
 - `price[i]` (giá),
 - `bonus_score[i]` (điểm khuyến mãi/ưu tiên).
- Ngân sách giảm giá/giá trần cho combo: B (không vượt quá).

Mục tiêu:

- Chọn tập sản phẩm làm combo sao cho:
 - Tổng `price` $\leq B$.
 - Tổng `bonus_score` lớn nhất.

Đây là knapsack 0/1 (weight = price, value = bonus_score).

Yêu cầu 3.2:

1. `build_combo_dp_table(prices, scores, B)`
 - Xây bảng dp 2D: `dp[i][b]` – max score với i item, budget b .
2. `trace_combo_from_dp(dp, prices, scores, B)`
 - Truy vết item được chọn (index sản phẩm).
3. `demo_combo_knapsack_2d()`
 - Tạo 1 bộ sản phẩm thử (4–6 món).
 - In bảng dp (hoặc một phần), max score và tập sản phẩm được chọn.

3.3. Tối ưu memory: knapsack 1D cho budget lớn

Nếu B lớn (ví dụ 10^4), bảng 2D khá nặng.

Yêu cầu:

- Cài `combo_knapsack_1d(prices, scores, B)` với dp 1D, duyệt budget ngược.

`demo_combo_knapsack_1d()`:

- So sánh kết quả max score với phiên bản 2D.

main_assignment.py

Tạo menu console, ví dụ:

1. Demo routing – shortest path giữa 2 kho.
2. Demo MST – mạng kho tối thiểu.
3. Demo hash table đơn hàng.
4. Demo hashing: group anagrams, ngày liên tiếp, subarray sum = k.
5. Demo rolling hash tìm pattern trong log.
6. Demo DP cơ bản (fibonacci, climbing stairs).
7. Demo combo khuyến mãi (knapsack 0/1 – 2D, 1D).
8. Thoát.

Mỗi mục gọi hàm demo tương ứng, in kết quả giải thuật, có vài dòng giải thích ngắn cho user.

Sản phẩm nộp

Nén toàn bộ thư mục dự án thành file .zip:

- Tên: MSSV_Assignment_DSA.zip.
- Bên trong:
 - routing.py
 - hashing_tools.py
 - promo_optimizer.py
 - main_assignment.py
 - (Tùy chọn) tests/ – file test tự viết.
 - README.md mô tả ngắn gọn cách chạy từng demo.

Tiêu chí đánh giá

Y1: Nội dung và mức độ hoàn thành

1. Hoàn thành đầy đủ các chức năng chính đã mô tả (routing, hashing, DP/knapsack).
2. Các demo chạy không lỗi và cho kết quả đúng với test mẫu.

Y2: Kỹ năng vận dụng kiến thức & tư duy

1. Dùng đúng CTDL: graph + heap cho Dijkstra, DSU cho Kruskal, dict/set/hash cho bài hash, bảng/mảng cho DP.
2. Biết mapping bài toán thực tế vào mô hình giải thuật (graph, hash, dp).
3. Cách thiết kế input nhỏ minh họa (ví dụ network kho, combo sản phẩm) hợp lý và dễ hiểu.

Y3: Kỹ năng lập trình & tổ chức code

1. Code rõ ràng, chia module hợp lý, đặt tên biến/hàm thể hiện ý nghĩa.
2. Không lạm dụng copy-paste, biết dùng hàm chung (ví dụ DSU dùng lại cho nhiều nơi).
3. Có test/demo riêng cho từng phần, không trộn logic.

Y4: Trình bày & tuân thủ

1. Đúng format nộp bài, tên file, cấu trúc thư mục.
2. Tuân thủ quy định về trích dẫn, không gian lận.
3. Nếu có tham khảo code bên ngoài, ghi rõ nguồn (link) trong comment hoặc README.